

---

# SYNOPSIS

**Task Manager using CI/CD Pipeline with Auto Rollback feature**

---

**DevOps-Driven Task Manager with CI/CD Automation  
Using  
Docker, GitHub Actions, and Jenkins**

By: Akshat Raj

Reg.no: 12307113

Roll-no: 42

Section: 4OM60

---



---

**L** LOVELY  
**P** ROFESSIONAL  
**U** NIVERSITY

---

*Transforming Education Transforming India*

---

---

# TABLE OF CONTENTS

---

|                                                         |    |
|---------------------------------------------------------|----|
| <b>1. Introduction</b>                                  | 1  |
| <b>2. Problem Statement</b>                             | 2  |
| 2.1 Manual Deployment Risks                             | 2  |
| 2.2 Lack of Automated Validation                        | 2  |
| 2.3 Absence of Rollback Strategy                        | 2  |
| 2.4 Environment Inconsistency                           | 2  |
| 2.5 Scope of This Project                               | 3  |
| <b>3. Objectives</b>                                    | 4  |
| 3.1 Primary Objectives                                  | 4  |
| 3.2 Secondary Objectives                                | 4  |
| <b>4. Proposed System</b>                               | 5  |
| 4.1 System Architecture                                 | 5  |
| 4.2 Component Description                               | 5  |
| <b>5. Methodology</b>                                   | 6  |
| 5.1 Phase 1: Application Development                    | 6  |
| 5.2 Phase 2: Containerization with Docker               | 6  |
| 5.3 Phase 3: Continuous Integration with GitHub Actions | 6  |
| 5.4 Phase 4: Continuous Deployment with Jenkins         | 7  |
| 5.5 Auto Rollback Mechanism (deploy.sh)                 | 8  |
| <b>6. Tools &amp; Technologies</b>                      | 9  |
| 6.1 Frontend Technologies                               | 9  |
| 6.2 Backend Technologies                                | 9  |
| 6.3 Database                                            | 9  |
| 6.4 Containerization & Orchestration                    | 10 |
| 6.5 CI/CD Tools                                         | 10 |
| 6.6 Version Control & Collaboration                     | 10 |
| <b>7. Expected Outcome</b>                              | 11 |
| 7.1 Functional Application Outcomes                     | 11 |
| 7.2 CI Pipeline Outcomes                                | 11 |
| 7.3 CD Pipeline Outcomes                                | 12 |
| 7.4 Auto Rollback Outcomes                              | 12 |
| 7.5 Educational & Professional Outcomes                 | 12 |
| <b>8. Conclusion</b>                                    | 13 |
| <b>References</b>                                       | 14 |

---

## 1. Introduction

---

In the contemporary landscape of software engineering, the velocity of software delivery has become a decisive competitive advantage. Organizations worldwide are transitioning from traditional manual deployment cycles to fully automated pipelines that enable developers to deliver reliable software faster and with greater confidence. This transformation is driven by the DevOps philosophy — a culture that bridges the gap between Development (Dev) and Operations (Ops), emphasizing collaboration, automation, continuous monitoring, and rapid feedback loops.

This project presents the design and implementation of a full-stack Task Manager web application integrated with a modern CI/CD (Continuous Integration and Continuous Deployment) pipeline. The application provides essential task management capabilities including creating, viewing, updating, and deleting tasks, all persisted in a MongoDB database. The backend is built with Node.js and Express, while the frontend is developed using plain HTML, CSS, and JavaScript, making the project accessible and straightforward to understand for students at the undergraduate level.

What distinguishes this project from a standard web application is the robust DevOps infrastructure surrounding it. Every code change pushed to the main branch triggers an automated CI pipeline via GitHub Actions, which validates the code, checks the project structure, verifies JavaScript syntax, and builds a Docker image — all without human intervention. Upon successful CI validation, the Jenkins-based CD pipeline takes over, building a tagged Docker image, deploying it to the target environment, and performing automated health checks.

The most innovative feature of this project is the Auto Rollback Mechanism. In any real-world deployment scenario, a newly released version may fail to start correctly due to configuration errors, dependency mismatches, or environment-specific bugs. When such a failure is detected — specifically, when the application's /health endpoint fails to return HTTP 200 with the body 'OK' — the system automatically stops the failed container and restores the last known stable version. This zero-downtime recovery ensures that end users experience minimal or no service disruption.

The entire infrastructure is containerized using Docker and orchestrated with Docker Compose, ensuring environment consistency across development, testing, and production. This report documents the problem context, objectives, system architecture, methodology, technology stack, and anticipated outcomes of the project.

---

## 2. Problem Statement

---

Software development teams, particularly in academic and early-career professional environments, frequently encounter significant challenges when transitioning application code from a developer's local machine to a shared or production environment. These challenges arise from a combination of human error, inconsistent environments, and the absence of automated safety nets. The following problems are identified as the primary motivation for this project:

### 2.1 Manual Deployment Risks

Traditional deployment workflows require developers or system administrators to manually execute a series of commands on the target server — pulling the latest code, stopping the running service, updating dependencies, and restarting the application. This process is inherently error-prone. A single missed step, an incorrect command, or an incompatible environment variable can render the application non-functional, causing unplanned downtime that directly impacts users.

### 2.2 Lack of Automated Validation

Without a structured CI pipeline, code changes are deployed without any systematic validation. Syntax errors, missing files, broken imports, or misconfigured Docker images are discovered only after they reach the production environment. The absence of pre-deploy checks creates a reactive rather than proactive quality assurance posture, significantly increasing the cost of defect resolution.

### 2.3 Absence of Rollback Strategy

Even in organizations that have implemented basic automation, the critical question of 'what happens when a deployment fails?' often lacks a satisfactory answer. Most simple deployment scripts proceed with the deployment regardless of the application's actual health after startup. In the event of a failure, the operations team must manually identify the issue, revert the code, rebuild the image, and redeploy — a process that can take hours and results in extended service downtime.

### 2.4 Environment Inconsistency

A frequently cited developer frustration — 'it works on my machine' — stems from discrepancies between development, testing, and production environments. Differences in Node.js versions, MongoDB configurations, operating system libraries, and environment variables cause applications that function correctly locally to fail in production. Without containerization, reproducing and debugging such environment-specific failures is particularly time-consuming.

### 2.5 Scope of This Project

This project directly addresses the above challenges by implementing a containerized full-stack application with an end-to-end CI/CD pipeline and an automated rollback mechanism that eliminates manual intervention in both deployment and recovery scenarios. The solution is designed to be educational, demonstrating industry-standard DevOps practices within a constrained college lab environment.

---

## 3. Objectives

---

The primary and secondary objectives of this project are defined as follows, spanning both the functional application layer and the DevOps infrastructure layer:

### 3.1 Primary Objectives

#### O1. Develop a functional full-stack Task Manager application

Build a RESTful web application with create, read, update, and delete (CRUD) operations for tasks, persisted in a MongoDB database and served via a Node.js/Express backend with a responsive HTML/CSS/JavaScript frontend.

#### O2. Implement a Continuous Integration (CI) pipeline

Configure GitHub Actions to automatically run validation checks, verify JavaScript syntax, and build a Docker image on every push or pull request to the main branch, providing fast feedback to developers.

#### O3. Implement a Continuous Deployment (CD) pipeline

Configure a Jenkins pipeline to automatically deploy the application using Docker, running structured stages including code checkout, dependency installation, image build, container deployment, and health verification.

#### O4. Implement an Automated Rollback Mechanism

Develop a shell-script-based deployment system that stores the last stable Docker image tag and automatically reverts to the previous stable version if the newly deployed container fails to pass a /health endpoint check within the configured timeout period.

#### O5. Containerize the entire application stack

Use Docker and Docker Compose to encapsulate the Node.js backend and MongoDB database in isolated containers with a shared network, ensuring environment consistency and simplifying deployment across different host machines.

### 3.2 Secondary Objectives

- Demonstrate the practical integration of GitHub Actions (CI) with Jenkins (CD) in a student-level DevOps workflow.
- Provide a clear and reproducible project structure that can serve as a reference template for future DevOps projects.
- Showcase Docker health checks as an automated mechanism to verify deployment success before promoting a container to stable.
- Illustrate a blue-green inspired container naming strategy (taskmanager-backend-new vs. taskmanager-backend-stable) to enable zero-downtime rollback.
- Document the CI/CD architecture in a form suitable for academic evaluation and viva examinations.

---

## 4. Proposed System

---

The proposed system is a comprehensive DevOps-integrated web application that combines a functional task management interface with a production-grade automated deployment pipeline. The system is structured into three logical tiers: the Application Layer, the Containerization Layer, and the CI/CD Pipeline Layer.

### 4.1 System Architecture

The high-level architecture of the proposed system is described below. The system follows a layered design where each layer has clearly defined responsibilities:

| Layer / Component                    | Responsibility                                                                                  |
|--------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Frontend (HTML/CSS/JS)</b>        | Responsive SPA-style UI for task CRUD operations; served as static files by Express             |
| <b>Backend API (Node.js/Express)</b> | RESTful endpoints for task management; /health endpoint for deployment verification             |
| <b>Database (MongoDB/Mongoose)</b>   | Persistent document storage for task records; Mongoose schema validation                        |
| <b>Docker + Docker Compose</b>       | Containerization of backend and database; shared bridge network for inter-service communication |
| <b>GitHub Actions (CI)</b>           | Automated lint, syntax check, and Docker image build on every push to main                      |
| <b>Jenkins (CD)</b>                  | Automated deployment pipeline with six stages; triggers deploy.sh                               |
| <b>deploy.sh (Auto Rollback)</b>     | Shell script that deploys new container, verifies /health, and rolls back on failure            |
| <b>.deploy-state/ (State Store)</b>  | Flat-file storage of the current stable Docker image tag for rollback reference                 |

### 4.2 Component Description

**Task Manager Frontend:** A single-page application built with vanilla HTML, CSS, and JavaScript. The UI allows users to add new tasks with a title and description, view all existing tasks with their completion status, toggle tasks between completed and pending states, and delete tasks. The frontend communicates with the backend via Fetch API calls to the REST endpoints.

**Task Manager Backend (Node.js/Express):** The server-side application exposes five REST endpoints: GET /health for deployment monitoring, GET /api/tasks to retrieve all tasks, POST /api/tasks to create a new task, PATCH /api/tasks/:id to update a task's completion status, and DELETE /api/tasks/:id to remove a task. Mongoose middleware handles schema validation and error propagation.

---

**MongoDB Database:** A NoSQL document database storing task records with fields: title (String, required), description (String), completed (Boolean, default false), and createdAt (Date, auto-generated). The database runs in a dedicated Docker container with a named volume for data persistence across container restarts.

**Docker Containerization:** The backend is packaged in a Node.js Alpine Docker image for minimal footprint. Docker Compose orchestrates both the MongoDB and backend containers on a custom bridge network (taskmanager-network), enabling hostname-based communication (mongodb://mongodb:27017).

---

## 5. Methodology

---

The development methodology followed an iterative, milestone-based approach aligned with DevOps principles. The project was executed in four phases: Application Development, Containerization, CI Pipeline Setup, and CD Pipeline with Auto Rollback. Each phase was tested independently before integration into the unified pipeline.

### 5.1 Phase 1: Application Development

The Task Manager application was developed following the MVC (Model-View-Controller) architectural pattern. The Mongoose Task model defines the data schema. The Express controllers implement the business logic for each CRUD operation. The routes map HTTP methods to controller functions. The frontend's app.js manages DOM manipulation and communicates with the backend via the Fetch API.

#### Key development steps in this phase:

- Initialized Node.js project with Express, Mongoose, CORS, and dotenv dependencies.
- Defined the Task Mongoose schema with title, description, completed, and createdAt fields.
- Implemented five REST API endpoints and the critical /health endpoint.
- Developed the responsive frontend with real-time task list updates and error handling.
- Configured environment variables via .env for port and MongoDB URI settings.

### 5.2 Phase 2: Containerization with Docker

The application was containerized to achieve environment parity across development and production. A multi-stage-ready Dockerfile was created for the Node.js backend using the node:20-alpine base image to minimize the container footprint.

#### Containerization steps:

- Created backend/Dockerfile based on node:20-alpine; copies package files, runs npm install, and starts the server.
- Created docker-compose.yml defining two services: mongodb (mongo:7) and backend (custom build), connected via taskmanager-network.
- Configured Docker Compose health checks: MongoDB uses mongosh ping; Backend uses wget to call /health.
- Set depends\_on with condition: service\_healthy to ensure backend starts only after MongoDB is ready.
- Mapped MongoDB to host port 27018 to avoid conflicts with local MongoDB installations.
- Added a named volume (taskmanager-mongo-data) for MongoDB data persistence across restarts.

### 5.3 Phase 3: Continuous Integration with GitHub Actions

The CI pipeline was configured using GitHub Actions to automatically validate every code change before it can be deployed. The pipeline is defined in .github/workflows/ci.yml and is triggered on push and pull\_request events targeting the main branch.



---

**CI pipeline stages (executed sequentially):**

- Stage 1 — Checkout Code: actions/checkout@v4 clones the repository into the GitHub Actions runner environment.
- Stage 2 — Setup Node.js: actions/setup-node@v4 installs Node.js 20.x with npm dependency caching keyed to backend/package.json.
- Stage 3 — Install Dependencies: Runs npm install in the backend directory to resolve and install all listed npm packages.
- Stage 4 — Run Build Checks: Executes npm run check (scripts/check.js), which validates the existence of required project files and verifies structural integrity.
- Stage 5 — Verify Server Syntax: Runs node --check src/server.js to parse the JavaScript without executing it, catching syntax errors.
- Stage 6 — Docker Buildx Setup: Configures Docker Buildx for multi-platform build support.
- Stage 7 — Build Docker Image: Builds the Docker image tagged with the git commit SHA. The build itself validates the Dockerfile correctness.

---

## 5.4 Phase 4: Continuous Deployment with Jenkins

The CD pipeline is implemented as a declarative Jenkins Pipeline defined in the Jenkinsfile at the project root. Jenkins polls or receives a webhook trigger from GitHub after a successful CI run and executes six deployment stages:

- Stage 1 — Pull Code: checkout scm retrieves the latest source code from the configured Git repository.
- Stage 2 — Install Dependencies: Runs npm install in the backend directory within the Jenkins workspace.
- Stage 3 — Run Checks: Mirrors the CI validation: npm run check and node --check src/server.js are executed.
- Stage 4 — Build Docker Image: Builds a Docker image tagged with the Jenkins BUILD\_NUMBER for traceability.
- Stage 5 — Deploy Container: Executes scripts/deploy.sh, passing environment variables. The deploy script contains the full rollback logic.
- Stage 6 — Health Check: Executes scripts/health-check.sh as a final verification step after deployment.

## 5.5 Auto Rollback Mechanism (deploy.sh)

The auto rollback mechanism is the cornerstone feature of this project. It is implemented as a Bash script (scripts/deploy.sh) that follows a blue-green inspired deployment strategy using two named Docker containers: taskmanager-backend-new (the container running the newly built image under evaluation) and taskmanager-backend-stable (the container running the last known-good image in production).

**The rollback algorithm executes the following steps sequentially:**

- Step 1 — Read Stable Tag: The script reads .deploy-state/stable\_tag.txt to determine which image tag was last promoted to stable. This tag is used as the rollback target if the new deployment fails.
- Step 2 — Ensure Docker Network: The script creates the Docker bridge network (taskmanager-network) if it does not already exist.
- Step 3 — Launch New Container: A new Docker container named taskmanager-backend-new is started on port 5000. An 8-second startup wait is applied before health checks begin.
- Step 4 — Health Check Loop (5 retries): The script attempts up to five health checks at 3-second intervals using curl, validating both the HTTP status code (200) and response body ('OK').
- Step 5 — Rollback on Failure: If all five health checks fail, the script stops and removes the new container, then restores the previous stable image. Jenkins marks the build as FAILED.
- Step 6 — Promote on Success: If the health check passes, the script stops the old stable container, removes the temporary new container, and starts the new image as taskmanager-backend-stable.

---

## 6. Tools & Technologies

---

The project leverages a carefully selected set of industry-standard tools and technologies that reflect current best practices in full-stack development and DevOps engineering. Each tool was chosen for its relevance to the specific problem it solves within the system.

### 6.1 Frontend Technologies

| Tool / Technology        | Purpose & Role                                                                                   |
|--------------------------|--------------------------------------------------------------------------------------------------|
| <b>HTML5</b>             | Markup language for structuring the task management user interface.                              |
| <b>CSS3</b>              | Stylesheet language providing responsive layout, card-based design, and visual feedback.         |
| <b>JavaScript (ES6+)</b> | Client-side scripting for dynamic DOM updates and Fetch API calls to the backend REST endpoints. |

### 6.2 Backend Technologies

| Tool / Technology     | Purpose & Role                                                                                                 |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Node.js 20 LTS</b> | JavaScript runtime for executing server-side code, chosen for its non-blocking I/O model and npm ecosystem.    |
| <b>Express.js 4.x</b> | Minimal web framework for Node.js providing routing, middleware support, and static file serving.              |
| <b>Mongoose 8.x</b>   | ODM (Object Document Mapper) library for MongoDB, providing schema definition, validation, and query building. |
| <b>dotenv</b>         | Module for loading environment variables from .env files, keeping sensitive configuration out of source code.  |
| <b>CORS</b>           | Express middleware enabling Cross-Origin Resource Sharing for frontend-to-backend API communication.           |

### 6.3 Database

| Tool / Technology    | Purpose & Role                                                                                                              |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>MongoDB 7.x</b>   | NoSQL document database storing task records as BSON documents; selected for schema flexibility and JSON-native data model. |
| <b>Docker Volume</b> | Named volume (taskmanager-mongo-data) providing persistent storage for MongoDB data across container lifecycle events.      |

---

## 6.4 Containerization & Orchestration

| Tool / Technology     | Purpose & Role                                                                                                                  |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Docker</b>         | Container runtime for packaging the Node.js application and its dependencies into portable, isolated execution environments.    |
| <b>Docker Compose</b> | Multi-container orchestration tool defining the full application stack (MongoDB + Backend) in a single docker-compose.yml file. |
| <b>Docker Buildx</b>  | Extended build capabilities for Docker, used in the GitHub Actions CI pipeline for multi-platform image builds.                 |
| <b>Alpine Linux</b>   | Minimal Linux distribution (node:20-alpine) used as the Docker base image to minimize image size and attack surface.            |

## 6.5 CI/CD Tools

| Tool / Technology           | Purpose & Role                                                                                                                              |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>GitHub Actions</b>       | Cloud-based CI automation service; triggers on push/PR to main, runs validation and Docker build steps defined in .github/workflows/ci.yml. |
| <b>Jenkins</b>              | Open-source automation server for CD; executes the declarative pipeline defined in Jenkinsfile across six structured stages.                |
| <b>Bash Shell Scripting</b> | Used to implement scripts/deploy.sh (auto rollback logic) and scripts/health-check.sh (standalone health verification).                     |
| <b>curl</b>                 | Command-line HTTP client used within deploy.sh to make health check requests and parse response codes.                                      |

## 6.6 Version Control & Collaboration

| Tool / Technology | Purpose & Role                                                                                                            |
|-------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Git</b>        | Distributed version control system for tracking source code history and enabling branch-based CI/CD triggers.             |
| <b>GitHub</b>     | Cloud-hosted Git repository platform providing source control, GitHub Actions CI integration, and pull request workflows. |

---

## 7. Expected Outcome

---

Upon successful completion of this project, the following outcomes are expected across the functional, operational, and educational dimensions of the system:

### 7.1 Functional Application Outcomes

- A fully functional Task Manager web application accessible via a browser at `http://localhost:5000`, with a responsive user interface.
- Ability to perform all CRUD operations (Create, Read, Update, Delete) on tasks through both the UI and REST API endpoints.
- Persistent task storage in MongoDB that survives backend container restarts due to named volume configuration.
- A reliable `/health` endpoint (`GET /health` → HTTP 200, body: 'OK') that accurately reflects the application's operational status.
- Correct JSON API responses with appropriate HTTP status codes (200, 201, 400, 404, 500) for all task endpoints.

### 7.2 CI Pipeline Outcomes

- Every git push to the main branch automatically triggers the GitHub Actions CI workflow within seconds.
- Code with syntax errors, missing required files, or invalid Dockerfile configurations will cause the CI pipeline to fail and block deployment.
- Developers receive visual pass/fail feedback via the GitHub Actions tab on the repository, with detailed logs for each stage.
- A Docker image tagged with the git commit SHA is successfully built during CI, confirming end-to-end build validity.
- The CI pipeline completes in approximately 3–5 minutes for standard code changes.

### 7.3 CD Pipeline Outcomes

- Jenkins successfully executes all six pipeline stages without manual intervention.
- Each Jenkins build produces a uniquely tagged Docker image (`taskmanager-backend:<BUILD_NUMBER>`) providing full deployment auditability.
- The deployed application is accessible on port 5000 immediately after a successful pipeline run.
- The `.deploy-state/stable_tag.txt` file is updated after each successful deployment, maintaining an accurate rollback reference.
- Jenkins console output and stage visualization provide clear deployment traceability for audit and debugging purposes.

### 7.4 Auto Rollback Outcomes

- When a deliberately broken deployment is triggered, the `/health` check fails, and the system automatically rolls back to the previous stable image within 30–60 seconds.
- The previous stable container (`taskmanager-backend-stable`) is restored and serving traffic on port 5000 within seconds of rollback completion.

- 
- Jenkins marks the failed build with a RED status, clearly communicating deployment failure to the team.
  - The .deploy-state/stable\_tag.txt file retains the last known-good tag and is not overwritten by a failed deployment.
  - The rollback process requires zero manual intervention from the operations team.

## 7.5 Educational & Professional Outcomes

Beyond the technical deliverables, this project provides the following learning outcomes for the student developer:

- Practical hands-on experience with GitHub Actions workflow syntax, job configuration, and step-level debugging.
- Understanding of Jenkins Declarative Pipeline syntax, agent configuration, environment variables, and post-condition blocks.
- Proficiency in Docker image building, multi-container orchestration with Docker Compose, and container networking.
- Experience writing production-quality Bash deployment scripts with error handling, retry logic, and state management.
- Appreciation for the importance of health checks as a deployment gate mechanism in real-world production systems.
- Understanding of blue-green deployment concepts as applied to container-based architectures.

---

## 8. Conclusion

---

This project successfully demonstrates the design, development, and deployment of a full-stack Task Manager web application underpinned by a complete CI/CD pipeline with an automated rollback feature. The system encapsulates core DevOps principles — automation, repeatability, rapid feedback, and resilience — within a project scope that is accessible to undergraduate Computer Science students yet technically rigorous enough to reflect industry standards.

The application itself fulfills all functional requirements: users can create, view, update, and delete tasks through an intuitive web interface backed by a RESTful Node.js API and a MongoDB database. All data is persisted across container restarts, and the API adheres to REST conventions with appropriate status codes and JSON response formats.

The CI pipeline, implemented via GitHub Actions, provides an automated quality gate on every code change. By validating project structure, checking JavaScript syntax, and building the Docker image before any deployment occurs, the pipeline prevents defective code from reaching production environments. This embodies the 'shift left' philosophy of catching errors as early as possible in the software delivery lifecycle.

The CD pipeline, powered by Jenkins, automates the entire deployment process from code checkout to container deployment. The structured six-stage pipeline provides clear visibility into each deployment step and maintains a consistent, repeatable process that eliminates human error from the deployment workflow. The unique tagging strategy — using Jenkins build numbers — creates a complete audit trail of every deployment attempt.

The Auto Rollback Mechanism represents the most significant technical contribution of this project. By implementing a health-check-gated deployment process with automatic reversion to the last stable Docker image, the system achieves a level of operational resilience that is typically found only in mature production environments. The blue-green inspired container naming strategy (new vs. stable) enables instantaneous traffic switchback without requiring load balancer reconfiguration.

Containerization with Docker and Docker Compose resolves the environment inconsistency problem comprehensively. The identical container image runs on any Docker-compatible host — eliminating the 'it works on my machine' class of deployment failures entirely.

### Future Enhancements

The following enhancements could extend the project in future iterations:

- Integration with a container registry (Docker Hub, GitHub Container Registry) to push and pull versioned images rather than building locally.
- Implementation of automated unit and integration tests using Jest or Mocha, making the CI pipeline truly test-driven.

- 
- Addition of a Kubernetes deployment manifest (Deployment + Service) to demonstrate rollout/rollback using `kubectl rollout undo`.
  - Slack or email notifications via Jenkins post-build actions to alert the team of deployment success or rollback events.
  - Implementation of a monitoring dashboard using Prometheus and Grafana to visualize application health metrics over time.
  - Multi-environment pipeline (dev → staging → production) with environment-specific configuration management.

In summary, this project provides a comprehensive, end-to-end demonstration of modern DevOps practices applied to a real-world web application. The combination of automated CI validation, structured CD deployment, and resilient auto rollback creates a software delivery system that is reliable, traceable, and production-worthy — embodying the fundamental goal of DevOps: delivering software faster and with greater confidence.



---

## . References

---

- [1] GitHub Inc., "GitHub Actions — Automate your workflow," GitHub Documentation, 2024. [Online]. Available: <https://docs.github.com/en/actions>
- [2] Jenkins CI, "Jenkins User Documentation — Pipeline Syntax," Jenkins.io, 2024. [Online]. Available: <https://www.jenkins.io/doc/book/pipeline/syntax/>
- [3] Docker Inc., "Docker Documentation — Dockerfile Reference," Docker Docs, 2024. [Online]. Available: <https://docs.docker.com/engine/reference/builder/>
- [4] Docker Inc., "Docker Compose — Compose File Reference," Docker Docs, 2024. [Online]. Available: <https://docs.docker.com/compose/compose-file/>
- [5] MongoDB Inc., "MongoDB Documentation — Introduction to MongoDB," MongoDB Docs, 2024. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [6] Mongoose ODM, "Mongoose v8.x Documentation," Mongoosejs.com, 2024. [Online]. Available: <https://mongoosejs.com/docs/>
- [7] Node.js Foundation, "Node.js v20 LTS Documentation," Nodejs.org, 2024. [Online]. Available: <https://nodejs.org/en/docs/>
- [8] Express.js, "Express 4.x API Reference," Expressjs.com, 2024. [Online]. Available: <https://expressjs.com/en/api.html>
- [9] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley, 2010.
- [10] G. Kim, P. Debois, J. Willis, and J. Humble, The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations. IT Revolution Press, 2016.
- [11] M. Burns, J. Beda, and K. Hightower, Kubernetes: Up and Running — Dive into the Future of Infrastructure, 3rd ed. O'Reilly Media, 2022.
- [12] GNU Project, "Bash Reference Manual," GNU.org, 2023. [Online]. Available: <https://www.gnu.org/software/bash/manual/>

---

— End of Report —